BIT

SHSID CPC

What are Binary Index Trees and Segment Trees?

Both data structures are used to:

1. Increment the value of one (or multiple, but that’s slightly harder) node by some value,
2. Find the sum of the values in some subarray,

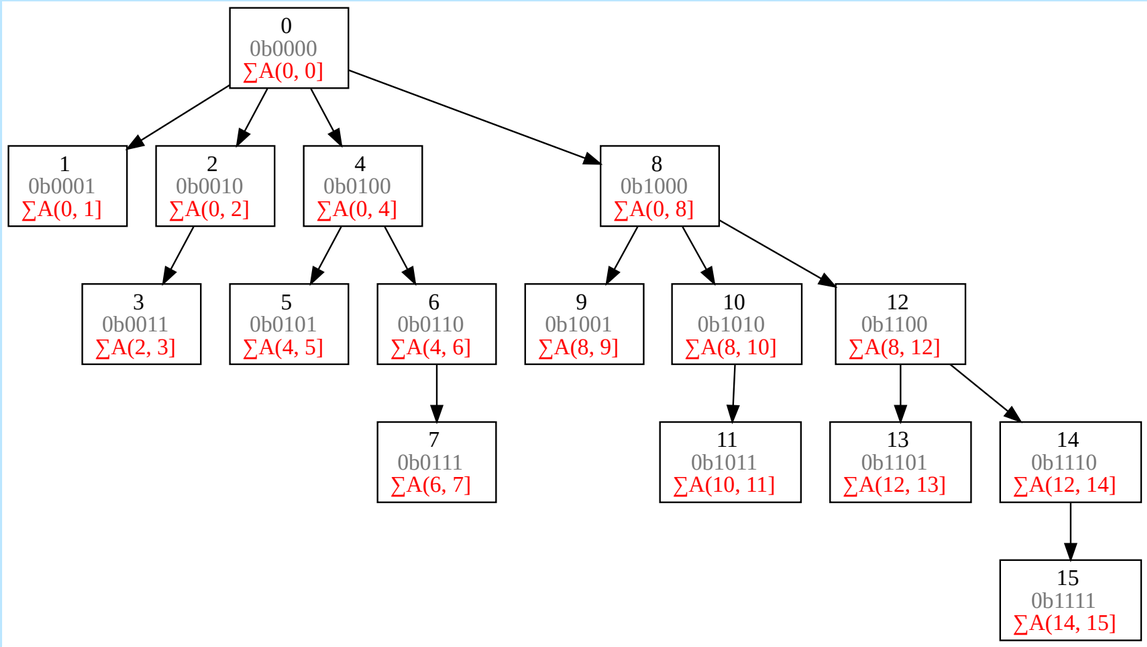
both in O(logn) time.

What is a BIT?

A tree that utilizes “lowest bit” to get a range sum/point update in log(n) time

Lowbit(x) is the lowest value 1 in the binary representation of x, calculated as x&(-x)

Pro: easy to write, small constant factor
Con: not too flexible, could not handle tasks like range max



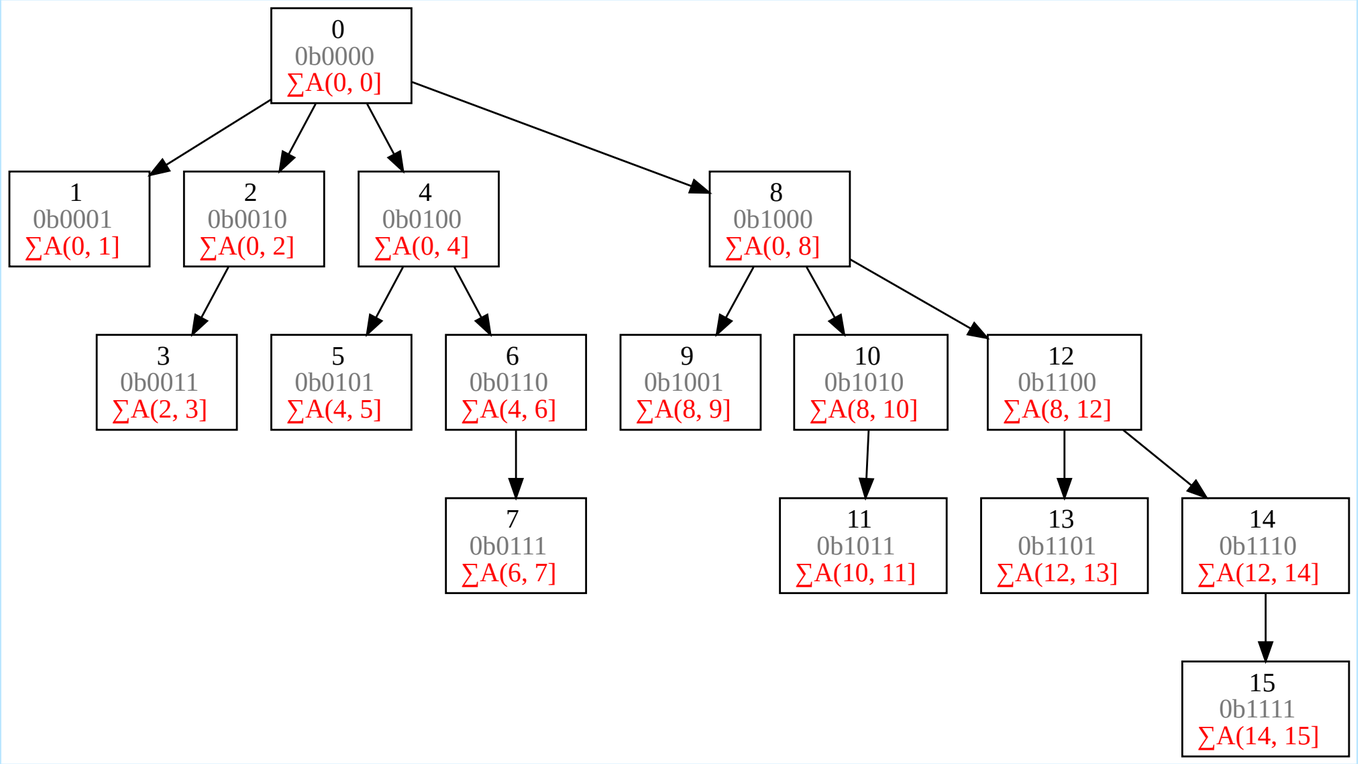
A 16-node BIT (for 1-15)

What is a BIT?

For a prefix sum (say to x): Start with node x on the tree. Observe that by adding the values of the nodes going up the tree to the root, you get the prefix sum of the first x nodes

→ a basic recursion:

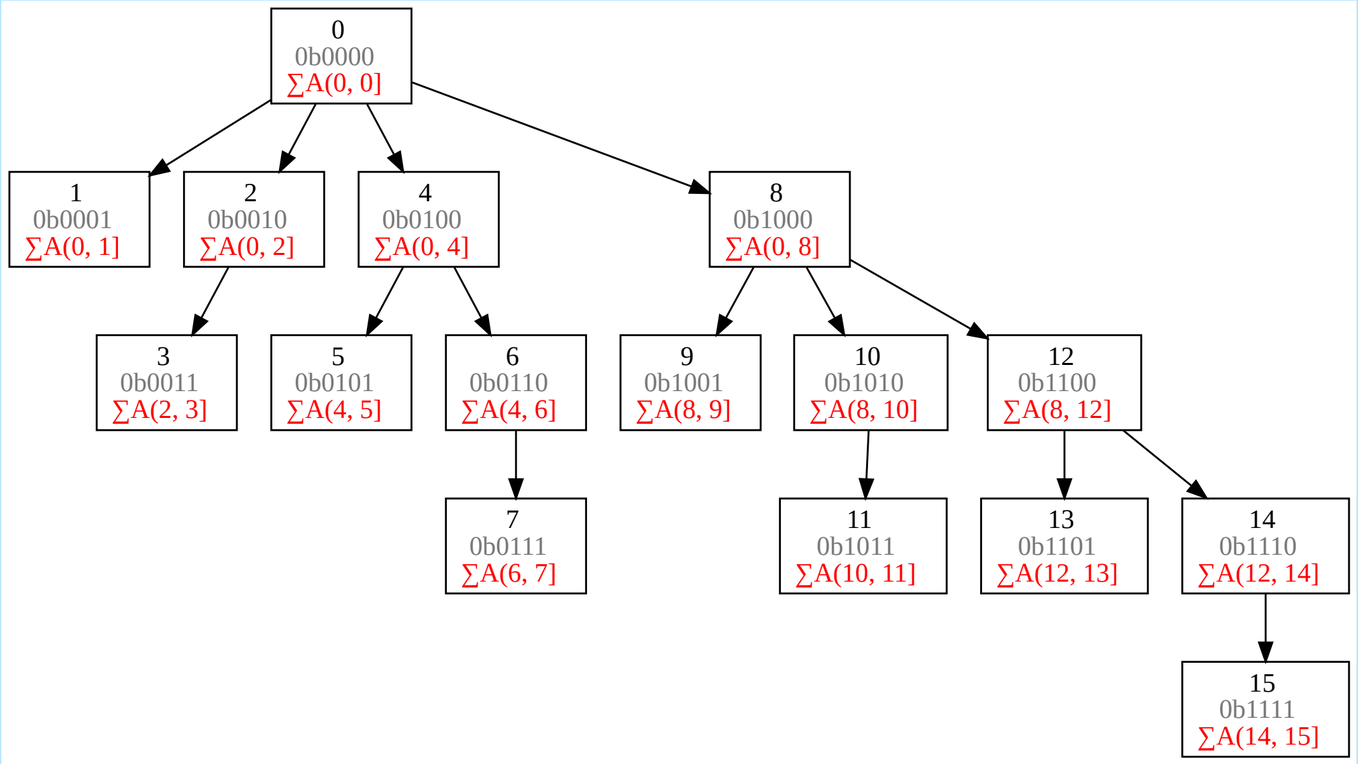
```
ans = 0
while x > 0:
    ans += fenwick[x]
    x -= (x&(-x))
return ans
```



What is a BIT?

For an update: we want to add d to node x (in our array). Thus, we need to add d to all the nodes on the tree that contain x. We get a recursion by observation:

```
while x <= n:
    fenwick[x] += d
    x += (x & (-x))
```



```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N = 200005;
#define lowbit(x) x&(-x)
int n,fenwick[N];
void add(int x, int d){
    for(int i = x; i <= n; i += lowbit(i)){
        fenwick[i] += d;
    }
}
int query(int a){
    int tot = 0;
    for(int i = a; i; i -= lowbit(i)){
        tot += fenwick[i];
    }
    return tot;
}
```

```cpp
signed main(){
    int arr[6] = { [0]: 0, [1]: 1, [2]: 2, [3]: 3, [4]: 4, [5]: 5};
    note that element 0 is not used
    n = 5;
    for(int i = 1; i <= n; i++){
        add( x: i,  d: arr[i]);
        initialization takes O(NlogN), which is sometimes annoying
    }
    for(int i = 1; i <= 10; i++){
        int a, b;
        cin >> a >> b;
        add( x: a,  d: b);
    }
    for(int j = 1; j <= 10; j++){
        int a,b;
        cin >> a >> b;
        cout << query( a: b) - query( a: a - 1) << '\n';
        typical prefix sums operation
    }
    return 0;
}
```